Bus Çeşitleri

FSB (Front Side Bus): İşlemci ve anakart arasındaki bağlantı.

Bellek Bağlantı Yolları: SDRAM, DDR-SDRAM, DDR2-SDRAM . . .

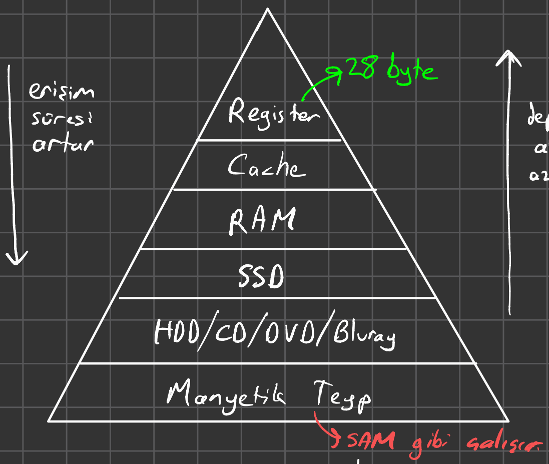
Grafik Birim Bağlantı Yolları (ekran kartı): AGP (Advanced Graphic Port), PCIe

İkincil Bellek Birim Bağlantı Y: ATA, SATA

Diğer Çevre Birim Bağlantı Y:

USB

Bellek Pramiti


erişim süresi artar
Register
28 byte
Cache
RAM
SSD
HDD/CD/DVD/Bluray
Manyetik Teyp
depolama alanı azalıyor
SAM gibi çalışır

SAM (Sequential)'in RAM'den farkı bir veriye erişmek için onun önündeki bütün verilere de erişmek zorundayız.

2^{10} → kilo → K

2^{20} → Mega → M

2^{30} → G

2^{40} → T

Ör: $2^{16} \to 65536$ $2^{16} \to 2^6 . 2^{10} \to 64K$

Ör: $2^{32} \to 2^2 . 2^{30} \to 4G$

Fiziksel Adres Hesabına Bak

MUL Op (unsigned/pozitif)

32 bit çarpma sınavda çıkacak!

$\text{MUL } \underbrace{CL}_{8\text{ bit}} \to AX \leftarrow \underbrace{AL}_{\text{gizli operand}} * CL$

$\text{MUL } \underbrace{CX}_{16\text{-bit}} \to \underbrace{\overset{\text{high}}{DX} : \overset{\text{low}}{AX}}_{32\text{ bit}} \leftarrow \underbrace{AX}_{\text{gizli operand}} * CX$

Ör:
```
MOV AL,17
MOV CL,2
MUL CL
```
↓

AX: 34
AH: 0
AL: 34
} son hali büyük olur

IMUL Op (signed)

Op ↓
RB
RW
DADDR

DIV Op
IDIV Op

Byte bölmesi

gizli operand

$$\begin{array}{r|l} AX & BL \text{ operant (8bit)} \\ \hline - & AL \\ AH & \end{array}$$

Sadece AX'i set ederek bölme yapma. Çünkü DX'in içindeki değer etki eder.

Word Bölmesi

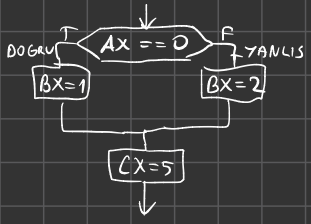
gizli operatör

$$\begin{array}{r|l} DX:AX & SI \text{ operand (16-bit)} \\ \hline - & AX \\ DX & \end{array}$$

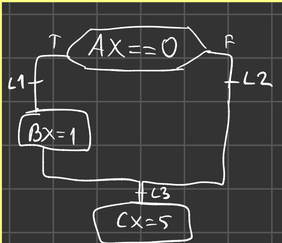
Dallanma Komutları

- Koşulsuz D.k.
  - → JMP
- Koşulu D.k. (JXXX)
  - Basit koşullu D.k.
    - → JZ/JE   (JZ ≡ JE)
    - → JNZ/JNE
    - → JS → sign
    - → JNS
    - → JO → overflow
    - → JNO
    - → JP/JPE
    - → JNP/JPO
  - Complex koşullu D.k
    - Unsigned Numbers (above/below)
      - → JB/JNAE/JC   (jump carry; Jump not above equal)
      - → JA/JNBE
      - → JAE/JNB/JNC
      - → JBE/JNA
    - Signed Numbers (greater/less)
      - → JL/JNGE
      - → JG/JNLE
      - → JGE/JNL
      - → JLE/JNG

! Zıplarken yukarı doğru max 128 byte, aşağı doğru 127 byte gidebiliriz. Fazlası için aralara koşulsuz zıplamalar koyup arttırabiliriz.


AX == 0
T
DOGRU
F
YANLIS
BX=1
BX=2
CX=5

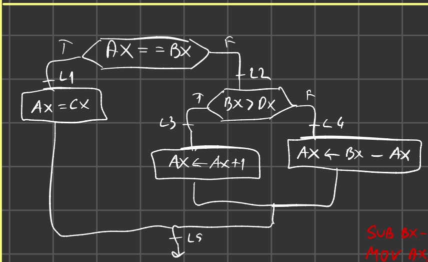
```
≡
CMP AX,0
JE   DOGRU
YANLIS: MOV BX,2
        JMP ORTAK
DOGRU: MOV BX,1
ORTAK : MOV CX,5
≡
```


AX==0
T
F
L1
L2
BX=1
L3
CX=5

```
≡
~~CMP AX,0~~
~~JE  L1~~
~~JMP L3~~
~~L1: MOV BX 1~~
~~L3: MOV CX 5~~
≡
```

tercih etme (ama doğru)

```
≡
CMP AX,0
JNE L3
MOV BX,1
L3: MOV CX,5
≡
```


AX==BX
T
F
L1
L2
AX=CX
BX>DX
T
F
L3
L4
AX←AX+1
AX←BX – AX
L5

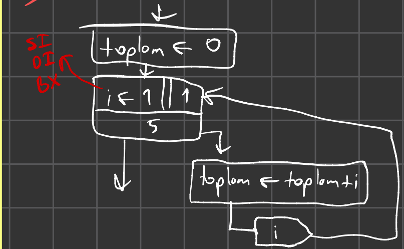
```
≡
CMP AX,BX
JNE L2
MOV  AX,CX
JMP L5
L2: CMP BX,DX
JG L3
SUB AX,BX
NEG AX
JMP L5
L3: INC AX
L5: ≡
```

SUB BX-AX
MOV AX,BX
olmaz çünkü BX ilk değerini koruyamadı

LOOP

Ör

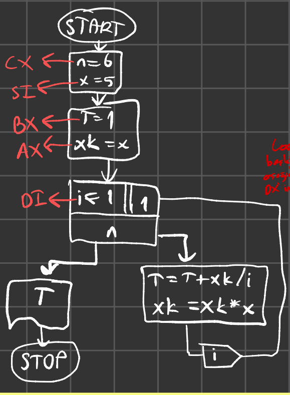
CX register çevrim değeri


SI
DI
BX
toplam ← 0
i ← 1 || 1
5
toplam ← toplam+i
i

```
≡
MOV toplam,0
MOV CX,5
MOV SI,1
L1: ADD  toplam,SI ; type misnatch
INC SI
LOOP L1 ; DEC CX  CMP CX>0
```

Not: Loopdan çıkıldığında CX 0 olur. Her iterasyonda 1 azalır. Çıkma koşulu CX > 0'dır.

Ör: $x = [1\text{-}5]$, $n = [1-6]$ olmak üzere $T = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} \dots + \frac{x^n}{n}$ ise T kaçtır?

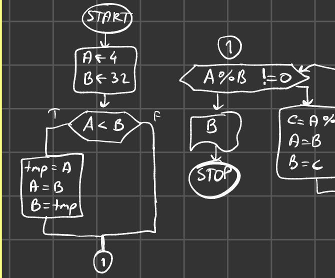

START
CX
SI
n=6
x=5
BX
AX
T=1
xk=x
DI
i ← 1 || 1
n
T
STOP
T=T+xk/i
xk=xk*x
i

Loop'u burdan başlatmadık çünkü aşağıda çarpmada DX'in 0 olması garanti

```
≡
MOV CX,6 ; MOV CX,n de olur.
MOV SI,5
MOV BX,1
MOV AX,SI
MOV DI,1
MOV DX,0
L1: PUSH AX
DIV DI
ADD BX,AX
POP AX
MUL SI
INC DI
LOOP L1
```

$$\begin{array}{r|l} DX:AX & DI \\ \hline - & AX \\ DX & \end{array}$$

Ör: A ve B = [0-5000] ise EBOB(A,B) = ?


START
A←4
B←32
A<B
T
F
tmp=A
A=B
B=tmp
1
1
A%B !=0
B
STOP
C=A%B
A=B
B=C

```
≡
MOV AX,4
MOV BX,32
CMP AX,BX
JAE L1
XCHG AX,BX ; ~~XCHG BX,AX~~
L1: MOV DX,0
DIV BX ; DX:AX | BX
CMP DX,0
JE L2
MOV AX,BX
MOV BX,DX
JMP L1
L2:
```

Adresleme Kipleri:

∝ Anlık (Immediate Adres):

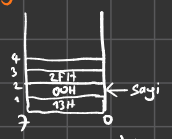
MOV {reg / data}, idata

```
MOV AX,17
```

∝ Yazmaç (Register) Adresleme

```
MOV reg, reg
MOV AX,SI
MOV DH,CL
```

∝ Doğrudan (Direct) Adresleme:



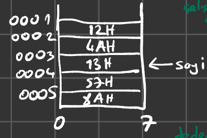
Not: sayı → word ise

```
MOV AX,sayi ; AX:2F00H
```

sayı → byte ise

```
MOV BH,sayi ; BH:00H
```

∝ Yazmaç Dolaylı (Register indirect) Adresleme



çalışma esnasında alır

```
LEA SI,sayi ; SI:0003H
MOV AX,[SI] ; AX:5713H
```

çok önemli !

derleme esnasında alır.

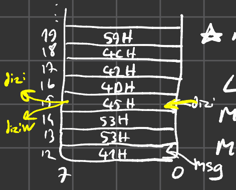
```
MOV BH,[SI] ; BH:13H
```

veya

```
MOV DI, OFFSET sayi
    ; DI:0003H
MOV DI,[DI] ; DI:5713H
```

LEA SI, DX
olmaz çünkü registerların
adresi yoktur

∝ Baz Göre (Base Relative) Adresleme



★ msg ← 'assembly'

```
LEA BX,msg
MOV AL,[BX+4] ; AL:40H
MOV CX,[BX+6] ; CX:594CH
```

∝ Doğrudan İndisli (direct index):

dizinin herbir elemanı byte tanımlı

```
XOR SI,SI
MOV AL,dizi[SI] ; AL:45H
INC SI
MOV BL,dizi[SI] ; BL:40H
```

```
XOR DI,DI
MOV AX,diziW[DI] ; AX:4045H
ADD DI,2
MOV CX,diziW[DI] ; CX:4C42H
INC DI
MOV CX,diziW[DI] ; CX:4240H
```

~~LEA SI,dizi ; SI:0015H~~ → ✗
~~MOV BH,dizi[SI]~~ → dizinin 21. elemanına gider

```
LEA SI,dizi
MOV BH,[SI]
```

→ ✓

∝ Dizgi (String) Adresleme

∝ İskele (Port) Adresleme

Ör: 2 dizinin yerini farklı adresleme kipleri kullanarak değiştirelim.

d1 → {12,15,-7,6,3}
d2 → {01H,01010101b,30,12,-17}
byte,n

```
XCHG d1[SI],[d2]
```

mem DADDR, mem DADDR
böyle bir komut yok !

~~MOV AL,[DI]~~
~~XCHG d1[SI],AL~~
~~MOV [DI],AL~~
AL'nin ilk değeri lazım (clock cycle)

```
MOV BL,d1[SI]
XCHG BL,[DI]
MOV d1[SI],BL
```

yine hatalı

```
    MOV CX,n
    XOR SI,SI ; d1
    LEA DI,D2 ; d2
L1: MOV AL,d1[SI]
    XCHG AL,[DI]
    MOV d1[SI],AL
    INC DI
    INC SI
    LOOP L1
```

Sözde (Pseudo) komutlar:

Segment / Ends:

```
<kesimismi> SEGMENT {seçenekler}
            ||||||
<kesimismi> ENDS
```

Seçenekler

a) Hizalama (alignment) tipi:

- →BYTE /1
- →WORD /2
- →PARA /16
- →PAGE /256

ile tam bölünmesi lazım

b) Birleştirme (combine) tipi:

- →PUBLIC
- →COMMON
- →STACK
- →AT ####H

c) Sınıf tipi:

PUBLIC combine tipindeki segmentleri birleştirmek için aynı sınıf ismine sahip olması gerekir. Tırnak içinde yazılır.

Ör:

```
stacksg SEGMENT PARA STACK 'yigin'
        ≡
stacksg ENDS
codesg SEGMENT WORD 'kod'
        ≡
codesg ENDS
datasg SEGMENT BYTE AT 34ABC 'data'
        ≡
datasg ENDS
```

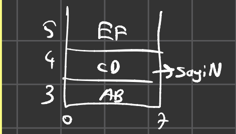
ASSUME

```
ASSUME SS:stacksg, DS:datasg, CS:Codesegment
```

DB (Define Byte)

```
Sayi1 DB 13              str1 DB 'Furkan'
Sayi2 DB ?
Sayi3 DB 01011011b
dizi1 DB 13,14,15,16
```

DW (Define word)



```
sayiW DW 0EFCDH
str2  DW 'furkan' ; 3word kaplar
```

DD (define double word) → 4 byte
DQ → 8 byte
DT → 10 byte

DUP (duplication factor)

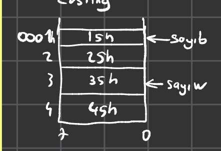
```
→DiziByte DB 15 DUP(2)
→DiziW    DW 10 DUP(?)
→mb1      DB 3 DUP(4 DUP(8))
```

→

8	8	8
8	8	8
8	8	8
8	8	8

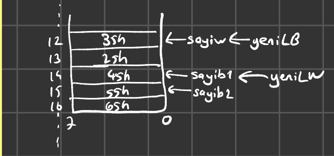
PTR (Pointer)

casting



```
sayib DB 15h
sayiw DW 4535h
MOV AX, WORD PTR sayib ; AX:2515h
MOV BL, BYTE PTR sayiw ; BL:35h
MOV BH, BYTE PTR sayiw+1 ; BH:45h
```

Label



```
yeniLB LABEL BYTE
sayiw  DW 2535h
yeniLW LABEL WORD
sayib1 DB 45h
sayib2 DB 55h
MOV AX, yeniLW ; AX:5545h
MOV sayiW, 0ABCDh
```

↓ memoryde tanımlanan ilk byte yeniLB olarak yazılır.

↓ tanımlanan ilk word yeniLW olarak yazılır.

→ yeniLB değeri değişir. ! (pointer mantığı)

PROC / ENDP

→ pseudo

```
<yordamadi> PROC {FAR|NEAR}
            ≡
            RET/RETF/IRET
<yordamadi> ENDP
```

→ mnemonic sınavda çıkar.

LENGTH

```
Tablo DW 15 DUP(?)
MOV AX, LENGTH Tablo ; AX:15
```

TYPE

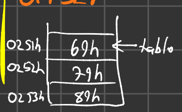
```
Tablo DD 15 DUP(?)
MOV AL, TYPE Tablo ; AL:4
```

SIZE

LENGTH * TYPE

```
MOV AX, SIZE Tablo ; AX:60
```

OFFSET



```
MOV AX, OFFSET Tablo
    ; AX ← 0251h
```

LEA ile aynı şeyi yapar.

SEG (Segment)

```
MOV ES, SEG Tablo
MOV CS, SEG TOPLAMA
```

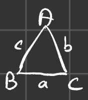
verdiğimiz ismin geçtiği segmentin adresini verir.

END

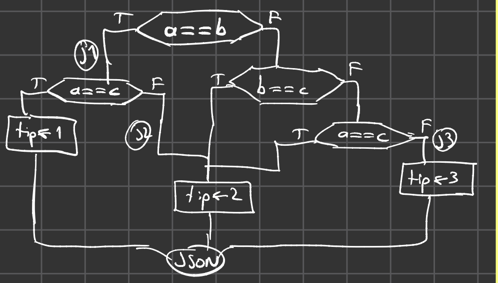
<başlangıçyordamıadı>
↓
istediğimizden başlatabiliriz.

Ör: 3 kenarı bilinen üçgenin tipini bulan algoritmanın assembly exe tipi programını yazalım.

tip ← 1 → eşkenar
tip ← 2 → ikizkenar
tip ← 3 → çeşitkenar



a , b , c
↓ ↓ ↓
AL BL CL


a==b
T
F
J1
a==c
T
F
tip←1
J2
b==c
T
F
a==c
T
F
J3
tip←2
tip←3
JSON

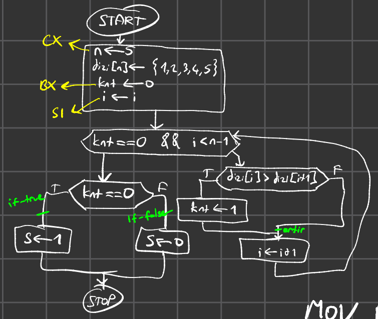
```
datasg  SEGMENT PARA 'veri'
 a    DB  12
 b    DB  17
 c    DB  12
 tip  DB  ?
datasg ENDS

stacksg SEGMENT PARA STACK 'yigin'
        DW  12  DUP(3)
stacksg  ENDS

codesg SEGMENT  PARA 'kod'
        ASSUME  cs:codesg, SS:stacksg, DS:datasg
UCGEN PROC FAR
        PUSH DS
        XOR  AX,AX
        PUSH  AX
        MOV  AX, datasg
        MOV  DS, AX
        MOV  AL, a
        MOV  BL, b
        MOV  Cl, c
        CMP  AL,BL
        JE J1
        CMP  BL,CL
        JE  J2
        CMP  AL,CL
        JE  J2
        MOV  tip,3
        JMP  JSON
J1:     CMP  AL,CL
        JNE  J2
        MOV  tip,1
        JMP  JSON
J2:     MOV  tip,2
JSON:   RETF
UCGEN   ENDP
codesg  ENDS
        END  UCGEN
```

Bu ilk beş satır fixdir.

Ör: Byte tanımlı bir dizinin küçükten büyüğe sıralı olup olmadığını kontrol eden EXE tipi asm programını yazalım.

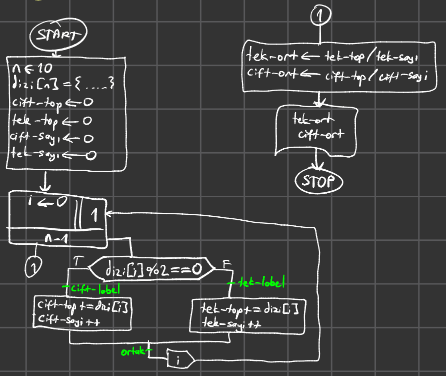

START
CX
n←5
dizi[n]← {1,2,3,4,5}
BX
knt ←0
SI
i←i
knt==0 && i<n-1
T
knt==0
F
if-true
if-false
S←1
S←0
T
dizi[i]> dizi[i+1]
F
knt ←1
artir
i←i+1
STOP

```
myds SEGMENT PARA 'veri'
 n DW 5
 dizi DB 1,2,3,4,5
 S  DB  ?
myds ENDS

myss SEGMENT PARA STACK 'yigin'
     DW  20  DUP(?)
myss ENDS

mycs SEGMENT PARA 'kod'
     ASSUME CS:mycs, DS:myds, SS:myss
MAIN  PROC  FAR
      PUSH  DS
      XOR  AX,AX
      PUSH  AX
      MOV  AX myds
      MOV  DS,AX
      XOR  BX,BX
      XOR  SI,SI
      MOV  CX,n
      DEC  CX
DONGU cmp  BX, 0
      JNE  son-if
      CMP  SI,CX
      JAE  son-if
      MOV  AL,dizi[SI]
      CMP  AL,dizi[SI+1]
      JLE  artir
      MOV  BX,1
artir: INC SI
      JMP  DONGU
son-if CMP BX,0
      JNE  sirasiz
      MOV  s,1
      JMP  son
sirasiz MOV s,0
son:  RETF
 MAIN  ENDP
mycs  ENDS
      END MAIN
```

Ör: Word tanımlı bir dizideki çiftlerin ve teklerin ortalamasını bulan EXE tipi programı yazınız.


START
n←10
dizi[n]={....}
cift-top←0
tek-top←0
cift-sayı←0
tek-sayı←0
i←0
1
n-1
1
T
dizi[i]%2==0
F
cift-label
tek-label
cift-top+=dizi[i]
cift-sayi++
tek-top+=dizi[i]
tek-sayi++
ortak
i
1
tek-ort← tek-top/tek-sayı
cift-ort← cift-top/cift-sayi
tek-ort
cift-ort
STOP

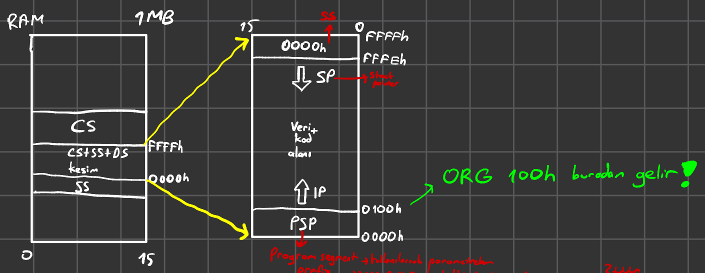
```
myds SEGMENT PARA 'v'
 n DW 10
 dizi DW  1,2,3,4,5,6,7,8,9,10
 tek-top DD  0
 cift-top DD  0
 tek-sayi DW 0
 cift-sayi DW 0
 tek-ort   DW  ?
 cift-ort  DW  ?
myds ENDS
myss SEGMENT PARA STACK 'stack'
     DW 20  DUP(?)
myss ENDS
mycs SEGMENT PARA  'code'
     ASSUME cs:mycs, ss:myss, ds:myds
ANA PROC  FAR
     PUSH DS
     XOR AX,AX
     PUSH AX
     MOV AX,myds
     MOV DS,AX
     MOV CX,n
     LEA SI,dizi
C1:  MOV  AX,[SI]
     TEST AX ,0001h
     JZ  cift-label
     ADD  WORD PTR [tek-top], AX
     ADC  WORD  PTR [tek-top+2], 0
     INC  tek-sayi
     JMP  ortak
cift-label: ADD WORD PTR [cift-top], AX
     ADC  WORD  PTR [cift-top+2],0
     INC  cift-sayi
ortak: ADD SI,2
     LOOP L1
     MOV DX,WORD PTR [tek-top+2]
     MOV AX,WORD PTR [tek-top]
     DIV tek-say
     MOV  tek-ort, AX
     MOV  DX,WORD  PTR [cifttop+2]
     MOV  AX,WORD PTR [cift-top]
     DIV  cift-sayi
     MOV  cift-ort, AX
     RETF
ANA  ENDP
myss ENDS
     END ANA
```

mnemonic ve pseudo code farkı ve nedir

↓ Sınav sorusu

- compact programlarda tüm segmentler tek bir segment altında toplanır.
- exe tipi programlarda her bir segment ayrı ayrı bir kesimde (segment) toplanır.

COM Tipi Asm Programı:



ORG 100h buradan gelir!

Program segment prefix → kullanılacak parametreler
→ program sonlandırıldığında hangi adrese dönülecek } tutulur
→ yardımcı aktarım değerlerinin tutulduğu alan

com tipinde segmentler yine ayrı ayrı tanımlanır ancak memoryde hepsi aynı yeri gösterir!

Syntax Model 1:

```
codesg SEGMENT PARA 'tum'
       ORG 100h  → kodun hangi offsetten başlayacağını söyler
       ASSUME CS:codesg, SS: codesg, DS: codesg
 ATLA  JMP BASLA
  dizi  DB 1,2,3,4
  yeni  DD   ?
   ⋮
  BASLA  PROC NEAR
   ⋮
    RET
BASLA ENDP
codesg ENDS
       END ATLA
```

Burada labellar ve RETF hariç her şey pseudo code'dur. RETF ise bir mnemonic'tir.

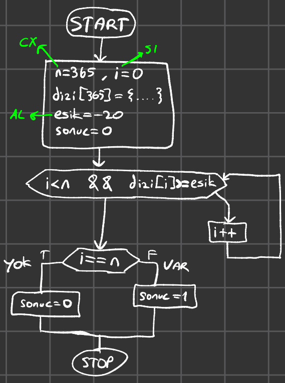
Syntax Model 2:

```
mys SEGMENT PARA 'hepsi'
    ORG 100h
    ASSUME cs:mys, ss:mys, ds:mys
ANA PROC NEAR
    ⋮
    RET
ANA ENDP
myveri DW  ?
dizi   DW  2 DUP (?)
 ⋮
mys    ENDS
       END ANA
```

Öncekinde değişken eklediğimizde yazdığımız kodların adresi değişir ve adrese dayalı jmp yapıyorsak kodda hatalar doğurabilir.

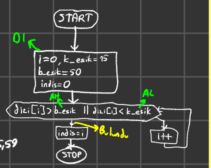
Ör: 1 yıl süresince günlük en düşük hava sıcaklığı bilgileri bulunan bir dizide -20'nin altında sıcaklık değeri olup olmadığını bulan com tipi ASM prog yazınız.
min sıcaklık → -90
max sıcaklık → 57

```
mys SEGMENT PARA 'hepsi'
    ORG 100
    ASSUME CS:mys, SS:mys, DS:mys
BASLA: JMP SICAKLIK
 n     DW   365
dizi   DB   365  DUP (?)
esik   DW   -20
Sonuc  DB   1
SICAKLIK PROC NEAR
         XOR SI,SI
         MOV CX,n
         MOV AL,esik
 ZIPLA:  CMP SI,CX
         JAE SON
         CMP AL,dizi[SI]
         JG VAR
         INC SI
         JMP ZIPLA
  VAR:   MOV sonuc,2
  SON:   RET
SICAKLIK ENDP
  mys    ENDS
         END BASLA
```



Ör: 15 yaşından küçük ve 50 yaşından büyük 10 tane kişinin olduğu sıraya yaş koşulunu bozan 11. kişi karışmıştır. Sıradaki tüm kişilerin yaşları bilindiğine göre, sıraya sonradan karışan kişinin kaçıncı sırada olduğunu bulan COM tipi ASM yaz.

```
mys  SEGMENT PARA 'tek'
     ORG 100h
     ASSUME cs:mys, ds:mys, ss:mys
KAYNAK PROC NEAR
      XOR DI,DI
      MOV AL, k_esik
      MOV AH, b_esik
ZIPLA: CMP dizi[DI], AH
      JA  SONRAKI
      CMP dizi[DI], AL
      JAE BULUNDU
SONRAKI: INC DI
         JMP ZIPLA
BULUNDU: MOV indis,DI
         RET
KAYNAK  ENDP
k_esik  DB  15
b_esik  DB  50
dizi    DB  12,6,1,6,7,19,51,63,79,55,59
indis   DW  ?
Mys     ENDS
        END KAYNAK
```

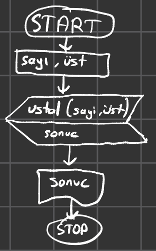


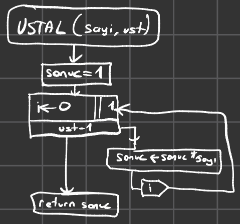
Yordamlar:

Public pseudo code'dur

```
<yordam-ismi>  PROC  {FAR|NEAR}
                ⋮
               RETF/RET
<yordam-ismi>  ENDP
```

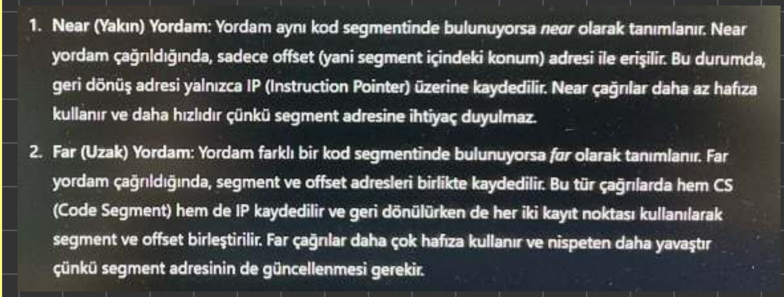
```
EXTRN  USTAL:FAR
codesg SEGMENT PARA 'kod'
USTAL  PROC  NEAR
        ⋮
        RET
USTAL ENDP
ANA  PROC  FAR
        ⋮
     CALL USTAL
        ⋮
       RETF
 ANA ENDP
codesg ENDS
       END ANA
```



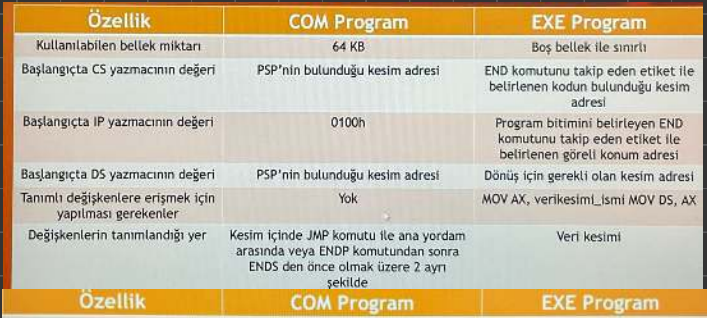


```
        PUBLIC USTAL
mycode  SEGMENT PARA 'kod'
        ASSUME cs: mycode
USTAL   PROC FAR
        MOV AX, 1
  L1:   MUL BX
        LOOP L1
        RETF
USTAL  ENDP
mycode ENDS
       END
```

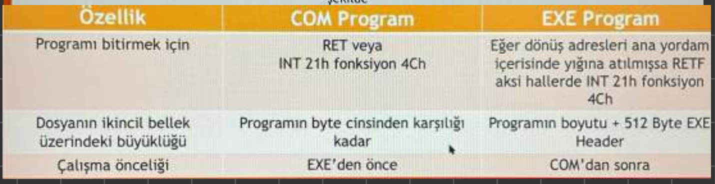
- Near procedure'den return yaparken RETF değil RET kullanılır.
- Exe tipi programda ana yordam FAR olarak tanımlanır.
- Com tipi programda ana yordam NEAR olarak tanımlanır.
- Com tipi programda geri dönüş adresleri yığına yerleştirilmez.



1. **Near (Yakın) Yordam:** Yordam aynı kod segmentinde bulunuyorsa *near* olarak tanımlanır. Near yordam çağrıldığında, sadece offset (yani segment içindeki konum) adresi ile erişilir. Bu durumda, geri dönüş adresi yalnızca IP (Instruction Pointer) üzerine kaydedilir. Near çağrılar daha az hafıza kullanır ve daha hızlıdır çünkü segment adresine ihtiyaç duyulmaz.
2. **Far (Uzak) Yordam:** Yordam farklı bir kod segmentinde bulunuyorsa *far* olarak tanımlanır. Far yordam çağrıldığında, segment ve offset adresleri birlikte kaydedilir. Bu tür çağrılarda hem CS (Code Segment) hem de IP kaydedilir ve geri dönülürken de her iki kayıt noktası kullanılarak segment ve offset birleştirilir. Far çağrılar daha çok hafıza kullanır ve nispeten daha yavaştır çünkü segment adresinin de güncellenmesi gerekir.



Özellik	COM Program	EXE Program
Kullanılabilen bellek miktarı	64 KB	Boş bellek ile sınırlı
Başlangıçta CS yazmacının değeri	PSP'nin bulunduğu kesim adresi	END komutunu takip eden etiket ile belirlenen kodun bulunduğu kesim adresi
Başlangıçta IP yazmacının değeri	0100h	Program bitimini belirleyen END komutunu takip eden etiket ile belirlenen göreli konum adresi
Başlangıçta DS yazmacının değeri	PSP'nin bulunduğu kesim adresi	Dönüş için gerekli olan kesim adresi
Tanımlı değişkenlere erişmek için yapılması gerekenler	Yok	MOV AX, verikesimi_ismi MOV DS, AX
Değişkenlerin tanımlandığı yer	Kesim içinde JMP komutu ile ana yordam arasında veya ENDP komutundan sonra ENDS den önce olmak üzere 2 ayrı şekilde	Veri kesimi



Özellik	COM Program	EXE Program
Programı bitirmek için	RET veya INT 21h fonksiyon 4Ch	Eğer dönüş adresleri ana yordam içerisinde yığına atılmışsa RETF aksi hallerde INT 21h fonksiyon 4Ch
Dosyanın ikincil bellek üzerindeki büyüklüğü	Programın byte cinsinden karşılığı kadar	Programın boyutu + 512 Byte EXE Header
Çalışma önceliği	EXE'den önce	COM'dan sonra

Not: Bir elemanı 2 ile çarparken 'SHR' kullanabiliriz ancak sayı negatifse bu yöntem hatalıdır. Bunun yerine 'SAR' kullanmalıyız.

Not: Makrolar derleme aşamasında, yordamlar çalışma aşamasında program tarafından çağrılır.

Yer çok fazla var ve daha hızlı çalışma isteği → makro
Yer kısıtlı ve işlem gücü çok problem değil → yordam

Not: Call ile harici bir yordam çağrıldığında code segment değerini ve offset değerini otomatik olarak atar.

Önemli Not: RETF 6 ile stack'den 5 tane word değer çeker.

3 + OFFSET + code segment = 5

MAKROLAR

```
cseg SEGMENT PARA 'ortak'
     ORG 100h
     ASSUME CS:cseg, SS:cseg, DS:cseg
ENKCK MACRO dizi, n
        LOCAL L1
        XOR SI, SI
        MOV AL, dizi[SI]
        INC SI
        MOV CX, n
        DEC CX
L1:     CMP AL, dizi[SI]
        JL don
        MOV AL, dizi[SI]
don:    INC SI
        LOOP L1
        ENDM
```

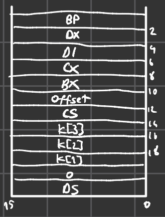
```
ANA   PROC NEAR
      XOR SI,SI
      MOV CX,n
L1:   SAR dizi[SI],1
      INC SI
      LOOP L1
      ENKCK dizi,n
      MOV kck, AL
      RET
ANA   ENDP
dizi  DB 10,-2,4,6,10,35,12
n     DW 0
kck   DB ?
cseg  ENDS
      END ANA
```

int → 4 bayt
short → 2 bayt
char → 1 bayt
Ancak adresler kaç bitlik mimari kullanılıyorsa o kadar bit ile tutulur. ❗

Ör: Ardışık 3 elemanı üçgen kenarı olan bir dizide n tane üçgenin kenarları tutulmaktadır. Üçgen kenarlarını stack üzerinden alan ve bu üçgenin alanının karesini AX register'ı üzerinden döndüren bir harici yordam kullanarak en büyük üçgenin alanının karesini bulan EXE tipi ASM yazınız.

$$u=(a+b+c)/2$$

$$A(\overset{\triangle}{ABC})=\sqrt{u\cdot(u-a)(u-b)(u-c)}$$

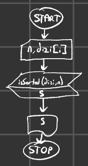


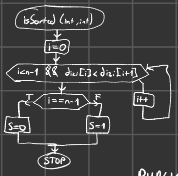
```
      EXTRN ALAN_BUL:FAR
myss  SEGMENT PARA STACK 'stack'
      DW 20 DUP(?)
myss  ENDS
myds  SEGMENT PARA 'd'
kenarlar DW 6,8,5,9,4,8,2,2,3
n        DW 3
enbykalan DW 0
myds  ENDS
mycs  SEGMENT PARA 'k'
      ASSUME CS:mycs, DS:myds, SS:myss
ANA   PROC FAR
      PUSH DS
      XOR AX,AX
      PUSH AX
      MOV AX,myds
      MOV DS, AX
      MOV CX, n
      XOR SI, SI
L1:   PUSH kenarlar[SI]
      PUSH kenarlar[SI+2]
      PUSH kenarlar[SI+4]
      CALL ALAN_BUL
      CMP AX, enbykalan
      JB kucuk
      MOV enbykalan, AX
      ADD SI,6
      LOOP L1
      RETF
ANA   ENDP
mycs  ENDS
      END ANA
```

133,38

```
         PUBLIC ALAN_BUL
Mycode   SEGMENT PARA 'kod'
         ASSUME CS:mycode
ALAN_BUL PROC FAR
         PUSH BX
         PUSH CX
         PUSH DI
         PUSH BP
         PUSH DX
         MOV BP, SP
         XOR AX, AX
         ADD AX, [BP+14]
         ADD AX, [BP+16]
         ADD AX, [BP+18]
         SHR AX, 1
         MOV BX, AX
         SUB BX,[BP+14]
         MOV CX, AX
         SUB CX, [BP+16]
         MOV DI, AX
         SUB DI, [BP+18]
         MUL BX
         MUL CX
         MUL DI
         POP DX
         POP BP
         POP DI
         POP CX
         POP BX
         RETF 6
ALAN_BUL ENDP
mycode   ENDS
         END
```

Ör: Verilen dizinin küçükten büyüğe sıralı olup olmadığını bulan harici yordamı yazınız. Harici yordam verilere EXTRN ve PUBLIC komutları ile erişmelidir.





```
      EXTRN isSorted:FAR
      PUBLIC dizi,n
Myss  SEGMENT PARA STACK 'sss'
      DW 20 DUP(?)
myss  ENDS
myds  SEGMENT PARA 'd'
n     DW 7
s     DB 0
dizi  DB 12,14,16,18,20,22,24
myds  ENDS
mycs  SEGMENT PARA 'k'
      ASSUME CS:mycs, SS:myss, DS:myds
ANA   PROC FAR
      PUSH DS
      XOR AX, AX
      PUSH AX
      MOV AX,myds
      MOV DS, AX
      CALL isSorted
      CMP AL,0
      JZ sirali
      MOV s,1
sirali: RET
ANA   ENDP
mycs  ENDS
      END ANA
```

```
         PUBLIC isSorted
         EXTRN dizi:BYTE, n:WORD
mycode   SEGMENT PARA 'test2'
         ASSUME CS:mycode
isSorted PROC FAR
         PUSH SI
         PUSH CX
         XOR AX,AX
         XOR SI,SI
         MOV CX,n
         DEC CX
don:     CMP SI,CX
         JAE sirali
         MOV AH,dizi[SI]
         MOV AH, dizi[SI+1]
         JG sirasiz
         INC SI
         JMP don
sirasiz: MOV AL
sirali : POP CX
         POP SI
         RET
isSorted ENDP
mycode   ENDS
         END
```

AX → 16 bit
EAX → 32 bit
RAX → 64 bit

32 bite geçildiğinde mnemonicler değişmiyor ancak pseudo komutlar değişecek.

section .data → data segment

derleme

```
nasm -f elf32 -o outputadi.o derlenecek.asm
```

link etme

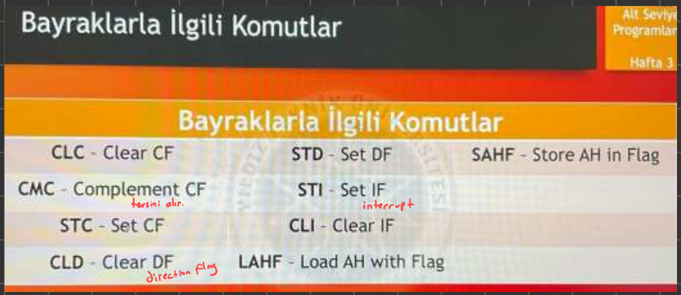
```
gcc -m32 151124.c asm_151124.o -o 151124
```

section .text → kod segmentin başlangıcını belirtir.

```
global Topla
  Topla:
      PUSH EBP
      PUSH EDI
      PUSH ECX
      MOV EBP,ESP
      MOV ECX,[EBP+20]
      MOV EDI,[EBP+16]
      XOR EAX,EAX
L1:   ADD EAX,[EDI]
      ADD EDI, 4
      LOOP L1
      POP ECX
      POP EDI
      POP EBP
      RET
```

Sayi → $\underbrace{1234}_{\substack{\text{high}\\ DX}}\ \underbrace{ABCD}_{\substack{\text{low}\\ AX}}h$ ⟹

```
MOV AX, WORD PTR[sayi]
MOV DX, WORD PTR[sayi+2]
```



Bayraklarla İlgili Komutlar

Alt Seviye Programlar
Hafta 3

Bayraklarla İlgili Komutlar

CLC - Clear CF	STD - Set DF	SAHF - Store AH in Flag
CMC - Complement CF tersini alır.	STI - Set IF interrupt	
STC - Set CF	CLI - Clear IF	
CLD - Clear DF direction flag	LAHF - Load AH with Flag	

α SHL

$\boxed{CF} \leftarrow$ işlenen $\leftarrow 0$

```
MOV AH,4
SHL AH,1   ;8
```

1'den fazla kaydırma yapılacaksa CL kullanılır

```
SHL AH,2     ✗
MOV CL,2  }
SHL AH,CL }  ✓
```